

17.10

Kubernetes Java API — Fabric8 Client

Build Operators & controllers in Java

Java Backend Architecture Interview Series • Tutorial Edition • Easy Diagrams & Examples

Why Interact with Kubernetes from Java?

Sometimes you need your Java application to talk to Kubernetes itself — to list pods, watch deployments, create resources dynamically, or build a Kubernetes Operator that automates custom resources. Fabric8 is the most popular Java client for this.

■ **Simple Analogy:** kubectl is like a TV remote — a human tool. Fabric8 is like a smart home hub — a program that talks to the TV on your behalf. Fabric8 lets Java code do everything kubectl can do, programmatically.

Fabric8 vs Official Java Client

Feature	Fabric8 Client	Official Client (client-java)
API style	Fluent DSL (chainable)	Generated, verbose
DSL readability	High	Low
Informer/cache	Yes (SharedIndexInformer)	Yes
Operator SDK	Yes (Java Operator SDK)	Limited
Community	Large	Smaller
Best for	Operators, controllers	Simple API calls

Basic Setup — Fabric8 Client

```
io.fabric8
kubernetes-client
6.10.0

// In-cluster (running inside K8s pod – uses mounted ServiceAccount token):
KubernetesClient client = new KubernetesClientBuilder().build();

// Out-of-cluster (local dev – uses ~/.kube/config):
KubernetesClient client = new KubernetesClientBuilder()
    .withConfig(Config.autoConfigure(null)).build();
```

Common Operations

```
// List all pods in a namespace
PodList pods = client.pods().inNamespace("production").list();
pods.getItems().forEach(p -> System.out.println(p.getMetadata().getName()));

// Watch pods (streaming – reacts to changes)
client.pods().inNamespace("production").watch(new Watcher() {
    public void eventReceived(Action action, Pod pod) {
        System.out.println(action + ": " + pod.getMetadata().getName());
    }
    public void onClose(WatcherException e) {}
});

// Scale a Deployment
client.apps().deployments()
    .inNamespace("production")
    .withName("java-api")
    .scale(5); // set replicas to 5

// Create a ConfigMap
ConfigMap cm = new ConfigMapBuilder()
    .withNewMetadata().withName("feature-flags").endMetadata()
    .addToData("DARK_MODE", "true")
    .build();
client.configMaps().inNamespace("production").create(cm);
```

Building a Simple Kubernetes Operator

An Operator is a Java application that watches custom resources and reacts to changes — automating operational tasks. The Java Operator SDK makes this easy.

```
@ControllerConfiguration
public class DatabaseOperator implements Reconciler {

    @Override
    public UpdateControl reconcile(
        Database resource, Context context) {

        String name = resource.getMetadata().getName();
        // React to a custom 'Database' resource being created/updated
        // e.g., provision a PostgreSQL instance automatically
        provisionPostgres(name, resource.getSpec().getSize());

        resource.setStatus(new DatabaseStatus("Ready"));
        return UpdateControl.patchStatus(resource);
    }
}
```

■ **Real-World Example:** The Strimzi Kafka Operator (written in Java with Fabric8) lets you declare 'I want a 3-broker Kafka cluster' as a YAML custom resource — and the Operator creates StatefulSets, Services, ConfigMaps, and manages rolling upgrades automatically. This is production Kubernetes at scale — 1000+ Kafka clusters managed this way.